



TEMASEK JUNIOR COLLEGE

2026 JC2 PRELIMINARY EXAMINATION

Higher 2



COMPUTING

9569/02

Paper 2 (Lab-based)

11 August 2026

3 hours

Additional Materials:

- Removable storage device
- Electronic version of `MEMBERS.txt` data file
- Electronic version of `FACILITIES.txt` data file
- Electronic version of `BOOKINGS.txt` data file
- Electronic version of `COMBATANTS.txt` data file
- Electronic version of `EQUIPMENT.txt` data file
- Electronic version of `STYLES.css` file
- Electronic version of `Task4_5_CLIENT.py` file
- Insert Quick Reference Guide

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

Note that up to **6** marks out of 100 will be awarded for the use of common coding standards for programming style.

The number of marks is given in brackets [] at the end of each question or part question.

The total number of marks for this paper is 100.

Instruction to candidates:

Your program code and output for each of Task 1 to 4 should be saved in a single `.ipynb` file. For example, your program code and output for Task 1 should be saved as:

`TASK1_<your name>_<centre number>_<index number>.ipynb`

Task 1

Name your Jupyter Notebook as:

`TASK1_<your name>_<centre number>_<index number>.ipynb`

A community sports centre intends to use a database to manage its facilities and member bookings. The sports centre offers various facilities, such as a swimming pool, badminton court and gym, which members can book for use.

Each booking is made by one member for one facility. A member may make multiple bookings, and a facility may be associated with multiple bookings.

The task is to implement the database for the community sports centre.

For each of the sub-tasks, add a comment statement, at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

In [1]: `# Task 1.1
Program code`

Output:

Task 1.1

Write a Python program that uses SQL to create a database called `sports_centre.db` with the following tables:

`Member (MemberID, FullName, ContactNumber, MembershipType)`

`Facility (FacilityID, FacilityName, PricePerHour)`

`Booking (BookingID, MemberID, FacilityID, BookingDate, DurationHours)`

Primary key attributes are indicated by a solid underline and foreign key attributes are indicated by a dashed underline.

Define all appropriate primary and foreign keys in the **three** tables.

[5]

Task 1.2

The sports centre currently stores its data in **three** separate text files: `MEMBERS.txt`, `FACILITIES.txt` and `BOOKINGS.txt`.

Each text file stores comma-separated values, with one record per line. The fields in the records appear in the same order as the attributes provided in the table descriptions in **Task 1.1**.

Write a Python program that uses SQL to insert the data from each of the **three** text files into the appropriate table in `sports_centre.db`.

Run your program to test that the data has been entered correctly.

[4]

Task 1.3

Write a Python procedure called `get_member_bookings(member_id)` that:

- takes a `member_id` as a parameter (assume that a valid `member_id` will be used)
- uses SQL to retrieve the details of all bookings made by that member
- outputs "No bookings found" if the member has no bookings
- otherwise outputs the `FullName`, `FacilityName`, `BookingDate`, and `DurationHours` for each booking, in descending order of `BookingDate`, in the format:
`<FullName> | <FacilityName> | <BookingDate> | <DurationHours> hour(s).`

Call the procedure with `member_id = "M001"` and show the output.

[5]

Task 1.4

Write a Python procedure called `get_total_cost(member_id)` that:

- takes a `member_id` as a parameter
- outputs an appropriate error message if no member with `member_id` exists in the database
- otherwise uses SQL to calculate the total cost of all bookings made by that member, where:
 $\text{Cost per booking} = \text{PricePerHour} \times \text{DurationHours}$
- outputs the result in the format:
`Total cost for <FullName>: <calculated total cost to 2 decimal places>`
- outputs "No bookings found" if the member has no bookings.

Call the procedure with `member_id = "M001"` and show the output.

[6]

Save your Jupyter Notebook for Task 1.

Task 2

Name your Jupyter Notebook as:

TASK2_<your name>_<centre number>_<index number>.ipynb

A prototype for a turn-based combat game needs to be created using Object-Oriented Programming and a priority queue implemented using a linked list.

The game has a player and a series of enemies. Each combatant has a cooldown that determines how frequently they can act. Combatants act in order of readiness where the one with the lowest `turns_before_ready` value acts first.

The task is to design and test the prototype for the game.

For each of the sub-tasks, add a comment statement, at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

In [1]:

```
# Task 2.1
Program code
```

Output:

Task 2.1

The class `Combatant` contains the following attributes:

- `name` — a string for the combatant's name
- `health` — an integer for the combatant's current health points
- `damage` — an integer representing the amount of damage the combatant deals per attack
- `cooldown` — an integer representing the number of turns between the combatant's actions

and methods:

- a constructor that takes `name`, `health`, `damage`, and `cooldown` as parameters
- `take_damage()` that takes an integer `points` as a parameter and decreases `health` by `points`
- `is_dead()` that returns `True` if `health` is less than or equal to 0, and `False` otherwise.

Write program code to declare the class `Combatant` and its methods.

[4]

Task 2.2

The class `Slot` represents a single node in the linked list and contains the following attributes:

- `combatant` — the `Combatant` object stored in this slot
- `turns_before_ready` — an integer initialised to the combatant's `cooldown` value, representing the number of turns that must pass before this combatant can act; this value must not fall below 0
- `next_slot` — a reference to the next `Slot`, initialised to `None`

and methods:

- a constructor that takes a `Combatant` object as a parameter and initialises all attributes appropriately
- `get_priority()` that returns `turns_before_ready` as the priority value of the node
- `update_priority()` that decrements `turns_before_ready` by 1 if its current value is greater than 0.

Write program code to declare the class `Slot` and its methods.

[4]

Task 2.3

The class `PriorityQueue` manages the linked list and contains the following attributes:

- `head` — a reference to the first `Slot` in the list, initialised to `None`
- `count` — an integer tracking the number of combatants in the queue, initialised to 0

and methods:

- a constructor that initialises `head` and `count` to their appropriate starting values
- `add()` that takes a `Combatant` as a parameter, creates a new `Slot` for it, then inserts the `Slot` into the linked list in ascending order of `turns_before_ready`; if existing `Slot(s)` have the same priority, the new `Slot` is inserted after all existing `Slots` with that priority; `count` is incremented by 1 upon successful insertion
- `remove()` that takes a `name` string as a parameter, then removes the `Slot` whose combatant has `name` from the linked list; `count` is decremented by 1 upon successful removal
- `dequeue_highest_priority()` that checks if the head `Slot` has a `turns_before_ready` value of 0 and if so, the head `Slot` is removed from the linked list, its `Combatant` is returned and `count` is decremented by 1; if the linked list is empty or the head `Slot` has a `turns_before_ready` value greater than 0, the method returns `None`
- `update_slots()` that calls `update_priority()` on every `Slot` in the linked list and outputs a message "Updating slots..."

Write program code to declare the class `PriorityQueue` and its methods.

[7]

Task 2.4

Two classes inherit from `Combatant`: `Player` and `Enemy`.

`Player` represents the human-controlled combatant and contains the method:

- `get_opponent_choice()` that takes a list of `Combatant` objects as a parameter, displays each with a numbered index starting from 1, prompts the user to input a number to choose an opponent, re-prompts until a valid integer index is entered, and returns the chosen `Combatant`.

`Enemy` represents a computer-controlled combatant and contains the method:

- `get_opponent_choice()` that takes a list of `Combatant` objects as a parameter and returns the first `Combatant` in the list that is an instance of `Player`; you may assume the list will always contain a `Player` object.

The text file `COMBATANTS.txt` stores data about all combatants in the game in comma-separated format. The first row is a header. Each subsequent row contains:

- `name` — the combatant's name; the value "player" identifies the human player
- `health` — an integer
- `damage` — an integer
- `cooldown` — an integer.

Write program code to:

- declare the `Player` and `Enemy` classes and their methods.
- read the data from `COMBATANTS.txt`
- create a `Player` object for the row where name is "player", and an `Enemy` object for all other rows
- add each `COMBATANT` to a global `PriorityQueue`. [6]

Task 2.5

When the game is played, combatants act in turn order determined by the priority queue.

On each turn, the following steps are carried out in order:

1. If `dequeue_highest_priority()` returns `None`, no combatant is ready — advance time by calling `update_slots()` on the queue.
2. Otherwise, the active combatant chooses an opponent from the queue.
3. The active combatant calls `get_opponent_choice()`, passing a list of all other combatants currently in the queue, to select an opponent.
4. The active combatant deals its damage to the chosen opponent using `take_damage()`, displaying an appropriate message, for example: "player dealt 8 damage to mosquito!"
5. If the opponent is `is_dead()`, display an appropriate message (e.g. "mosquito dies! ") and call `remove()` to remove the opponent from the queue.
6. Re-add the active combatant to the queue.

The game continues until the player is dead or is the only combatant remaining. After the game ends, print "You died!" if the player is dead, otherwise print "You won!".

Write program code for the game loop and test your program with the following inputs

- Input 1: mosquito
- Input 2: crow
- Input 3: crow
- Input 4: crow
- Input 5: rat
- Input 6: rat

[6]

Save your Jupyter Notebook for Task 2.

Task 3

Name your Jupyter Notebook as:

TASK3_<your name>_<centre number>_<index number>.ipynb

A community learning centre owns equipment that may be borrowed by its members.

The centre currently records equipment loans manually. To increase administrative efficiency, it has decided to develop a web application to manage its equipment records.

When a piece of equipment is purchased, the following information is recorded:

- `EquipmentID` – unique identifier of the piece of equipment
- `Name` – name and model of the piece of equipment
- `Category` – category of the piece of equipment
- `Location` – location where the piece of equipment is normally stored
- `Status` – current availability of the piece of equipment, either `AVAILABLE` or `ON_LOAN`
- `Borrower` – name of the person currently borrowing the piece of equipment or an empty string if it is currently available

A hash table containing 503 slots will be used to store and retrieve equipment records efficiently. The following hashing algorithm shall be used:

- use `EquipmentID` as the key for each record
- calculate the initial hash address as the remainder when the numerical part of `EquipmentID` is divided by 503 e.g. if the `EquipmentID` is `E1047`, the initial hash address will be `1047 MOD 503`

A suitable collision resolution algorithm will also be implemented to resolve all occurrences of collisions that occur when equipment records are inserted into the hash table.

The task is to develop a preliminary prototype for the web application with the following functionalities:

- view all equipment records
- search for a piece of equipment

For each of the sub-tasks, add a comment statement, at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: # Task 3.1
        Program code
```

Output:

Task 3.1

Write Python code to implement in either the procedural programming paradigm or object-oriented programming paradigm, a hash table containing 503 locations and its associated operations:

- `Hash(EquipmentID)`
 - extract the numerical part of `EquipmentID`
 - return the initial hash address as the remainder when the numerical part of `EquipmentID` is divided by 503.
- `Insert(Record)`
 - obtain the `EquipmentID` from the record
 - calculate the initial hash address
 - insert the record as a list as follows:
`[EquipmentID, Name, Category, Location, Status, Borrower]`
 resolving any occurrence of collision
 - return `True` when the record is inserted successfully, otherwise `False`.
- `Search(EquipmentID)`
 - calculate the initial hash address;
 - locate the record, taking into account any collision
 - terminate the search immediately if when an empty slot is accessed
 - return the location of the matching record when it is found, otherwise `-1`.

[7]

Task 3.2

The file `EQUIPMENT.txt` contains the existing equipment records.

The first row contains the field headers and each remaining row is a comma-separated list containing information about one equipment item in the following order:

```
EquipmentID,Name,Category,Location,Status,Borrower
```

Write Python code to:

- read all equipment records from `EQUIPMENT.txt`;
- insert each record into a hash table with 503 slots, storing the data using a suitable record structure
- display the number of records successfully inserted.

[4]

Save your Jupyter Notebook for Task 3

Task 3.3

The Cascading Style Sheets (CSS) file `STYLES.css` contains the CSS rules to be used by every page of the web application.

Write a Python program and the necessary HTML files to create a Flask web application that makes use of the hash table populated from `EQUIPMENT.txt` to implement the following functionalities from a navigation menu on the home page:

- view all equipment records
- search for a piece of equipment

To view all equipment records, the web application will:

- traverse the hash table
- display all equipment records in a HTML table in ascending order of `EquipmentID`.

To search for a piece of equipment, the web application will:

- allow the user to enter an `EquipmentID`.
- search for the piece of equipment in the hash table;
- display all the information for the matching piece of equipment or display an appropriate message when the piece of equipment is not found.

Save your Python program as:

`TASK3_3_<your name>_<centre number>_<index number>.py`

with any additional files and sub-folders in a folder named:

`TASK3_3_<your name>_<centre number>_<index number>.`

[7]

Task 3.4

Run the web application.

Display all the equipment records and save the output as:

`TASK3_4_1_<your name>_<centre number>_<index number>.html`

[1]

Test the search functionality of the web application by searching for the equipment with `EquipmentID: E1010` and save the output as:

`TASK3_4_2_<your name>_<centre number>_<index number>.html`

[1]

This page has been intentionally left blank

Task 4

Name your Jupyter Notebook as:

TASK4_<your name>_<centre number>_<index number>.ipynb

(0, 0)				(0, 4)
(4, 0)				(4, 4)

A computer game requires a player to move within the 5×5 grid shown above. Each location is represented by a tuple `(row, col)`, where `row` and `col` are the location's row and column coordinates respectively. Both `row` and `col` are integers in the range 0 to 4, inclusive.

The task is to implement the game using object-oriented programming and the actual gameplay on a client-server model.

For each of the sub-tasks, add a comment statement, at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: # Task 4.1
        Program code
```

Output:

Task 4.1

A player of the game can be modelled by the class `Player`, using the following attributes:

- `name` — a string representing the player's nickname
- `start_loc` — the player's starting location; has a constant value in tuple form `(0, 0)`
- `curr_loc` — the player's current location in tuple form `(row, col)`; initialised to `start_loc`
- `score` — an integer representing the player's current score; initialised to 0
- `turns_taken` — an integer representing the number of valid turns taken; initialised to 0
- `has_shield` — a Boolean representing whether the player currently has a shield; initialised to `False`

and methods:

- `constructor(nickname)` — constructor function for instantiating `Player` objects; takes in a string parameter `nickname` for the name attribute
- `update_score(points)` — adjusts the score attribute by the integer value `points`
- `reset_loc()` — resets the location of the player to `start_loc`
- `status()` — returns a string containing the location, score, shield and turns_taken attributes of the player, for example: `At (2, 3) | Score:10 | Turns:4 | Shield:True`
- `move(direction)` — receives a non-case sensitive direction from the player that will alter the `curr_loc` attribute as follows:
 - UP: from `(row, col)` to `(row-1, col)`
 - DOWN: from `(row, col)` to `(row+1, col)`
 - LEFT: from `(row, col)` to `(row, col-1)`
 - RIGHT: from `(row, col)` to `(row, col+1)`
 and moves the player if the resulting location is within the grid, returning `True`, or `False` otherwise.

Write code to implement the `Player` class.

[6]

Task 4.2

The grid contains objects that can be modelled by the class `GridObj`, using the attribute:

- `location` — the grid object's location in the form of a tuple `(row, col)`

and the method:

- `constructor(row, col)` — constructor function for instantiating `GridObj` objects at the start of the game, where `row` and `col` are generated from a list of available locations on the grid.

Write code to implement the `GridObj` class.

[2]

Task 4.3

Objects on the grid can be classified into four categories as follows:

Object	Symbol	Effect
Energiser	E	Adds 10 points to the player's score.
Shield	S	Gives the player one shield.
Toxin	T	Deducts 5 points from the player's score unless a shield is present, in which the shield will be destroyed instead.
Trap	X	Sends the player back to the starting location.

These categories of objects can be modelled by the `Energiser`, `Shield`, `Toxin` and `Trap` classes, using the `GridObj` class as a basis. Each of these classes have an additional attribute:

- `symbol` — a character used to represent the grid object, as shown in the table

and an additional method:

- `effect(player)` — which applies the effect shown in the table to the appropriate attribute of `player`.

Write code to implement the classes `Energiser`, `Shield`, `Toxin` and `Trap`, demonstrating appropriate inheritance and method overriding where appropriate.

[4]

Task 4.4

The gameplay can be modelled by the class `Game`, using the following attributes:

- `grid_size` — the size of the square grid, has a constant value of 5
- `player` — a `Player` object representing the player in the game
- `items` — a list that stores the grid objects in the game, initialised as an empty list
- `max_turns` — the maximum number of valid turns allowed, initialised to 8

and methods:

- `constructor(nickname)` — constructor function for instantiating `Game` objects; uses the string `nickname` to instantiate the `Player` object referenced by the `player` attribute
- `generate_items()` — randomly selects 6 locations on the grid to place grid objects, omitting the starting location `(0, 0)`; instantiates and stores the grid objects in the `items` attribute based on the following quantities:
 - Energiser: 2
 - Shield: 1
 - Toxin: 2
 - Trap: 1
- `process_move(direction)` — executes the `move(direction)` method of the `Player` object referenced by the `player` attribute; if the execution returns `True`:
 - increases the `turns_taken` attribute of the `Player` object by 1
 - checks the `items` attribute of the `Game` object to determine whether the new location contains a grid object and if it does:
 - applies the effect of the grid object to the `Player` object
 - removes the grid object from the `items` attribute of the `Game` object
 - returns `True` at the end of the process;
 if however the execution returns `False`:
 - output an appropriate message stating that an invalid move has been made.

Write code to implement the `Game` class, demonstrating the use of polymorphism where appropriate.

[7]

Save your Jupyter Notebook for Task 4.

Task 4.5

The code for the client that will be used for connecting to the game server is provided in the file `Task4_5_CLIENT.py`.

Using the client code provided, write code to implement a game server that allows one player to play the game. The game server code should be contained in the file:

`TASK4_5_<your name>_<centre number><index number>.py`

For this sub-task, you are to copy and paste all of the code you have written in **Task 4.1** to **Task 4.4** to `TASK4_5_<your name>_<centre number>_<index number>.py` before writing the code for the game server.

The game server should be implemented according to the following game rules:

1. The game will commence from the location `(0, 0)`.
2. The player may move one step per turn using the commands: `UP`, `DOWN`, `LEFT` or `RIGHT`, which are non-case sensitive.
3. The player cannot move out of the grid.
4. Invalid moves are rejected and do not count as a turn.
5. When the player visits a location containing an object, a gain or loss of points occurs, depending on the type of object at the location.
6. The game ends after the player has taken 8 valid turns.
7. The final score is displayed when the game ends.

In addition, the server code should include elements to:

- send the message `"WELCOME"` to the client
- send the message `"ENTER NAME"` to the client
- receive the player's nickname from the client
- instantiate a `Game` object using the player's nickname
- call the `generate_items()` method of the `Game` object to generate the grid objects
- send the message indicating the player's current status to the client
- send the message `"YOUR TURN"` to the client when a move is required
- receive a direction to move from the client
- process the move using the `process_move(direction)` method of the `Game` object
- continue the game until the player has taken 8 valid turns
- send the final score to the client when the game ends in the format:

`Game Over, <NAME OF CLIENT>! Final Score: <SCORE>`

[8]

Save your server program as:

`TASK4_5_<your name>_<centre number>_<index number>.py`

- End of Paper -

BLANK PAGE